

PRUEBA TÉCNICA

Desarrollador TI

Plataforma de gestión de pólizas de arrendamiento

Candidato Juan Pablo Restrepo B.

Contenido
Módulo 1 – Diseño de sistema
Módulo 2 – Solución práctica (resumen y repositorio)
Módulo 3 – Optimización de base de datos
Módulo 4 – Versionamiento con Git

Repositorio <https://github.com/JUANRESTREPO-DEV/polizas-api>

 CI **passing**

 codecov **92%**

 Java **17**

 Spring Boot **3.5**

Fecha 29 de septiembre de 2026

Contenido

0. Lectura crítica del enunciado y supuestos de diseño

1. Módulo 1 – Diseño de sistema

1.1 Arquitectura de alto nivel

1.2 Patrones de arquitectura seleccionados

1.3 Modelo de datos principal

1.4 Escalabilidad, observabilidad, tolerancia a fallos y versionamiento

1.5 Diagrama de componentes y flujo de renovación

2. Módulo 2 – Solución práctica: API de gestión de pólizas

3. Módulo 3 – Optimización de la consulta orders / customers

4. Módulo 4 – Incorporar un fix de seguridad de `main` en `feature/new-login`

0. Lectura crítica del enunciado y supuestos de diseño

Antes de diseñar revisé el enunciado buscando reglas incompletas o que, tomadas al pie de la letra, producirían comportamientos incorrectos en un sistema asegurador. Estos son los puntos encontrados y la decisión tomada en cada caso; todas están implementadas y probadas en el Módulo 2.

#	Observación	Decisión
1	El canon se define a nivel de póliza, pero una colectiva cubre N inmuebles, cada uno con un canon distinto. Un único canon para toda la colectiva no permite recalcular la prima al agregar o cancelar riesgos.	El canon vive en el riesgo . El canon de la póliza es la suma de los riesgos activos y la prima es $\text{canon} \times \text{meses}$. En la individual coincide con el de su único riesgo, así que la regla es la misma para ambos tipos.
2	"Incrementa canon y prima en +IPC": si se aplica el IPC por separado a canon y a prima, el redondeo puede romper la igualdad $\text{prima} = \text{canon} \times \text{meses}$.	El IPC se aplica al canon de cada riesgo activo y la prima se recalcula . El incremento porcentual resultante es el mismo y la invariante se conserva.
3	No se indica el valor del IPC ni su fuente.	Parámetro configurable (<code>polizas.renovacion.ipc-porcentaje</code> , por defecto 5,20). En producción se lee del servicio de parámetros alimentado con la cifra oficial del DANE.
4	"Renovar por el mismo periodo inicial" no dice desde cuándo corre la nueva vigencia.	La nueva vigencia inicia en la fecha fin de la anterior y dura el mismo número de meses. Cada renovación queda en <code>historial_renovacion</code> .
5	No se define qué pasa al cancelar el último riesgo activo. Permitirlo deja una póliza vigente con prima cero y sin objeto asegurado.	Se rechaza con 409 <code>ULTIMO_RIESGO_ACTIV0</code> : lo que corresponde es cancelar la póliza. Aplica también al único riesgo de una individual.
6	No hay restricción de ventana de renovación: el endpoint podría invocarse varias veces seguidas y aplicar el IPC de forma compuesta.	En la prueba se permite renovar una póliza <code>ACTIVA</code> o <code>RENOVADA</code> , tal como pide el enunciado. En producción se agrega una ventana (p. ej. 30 días antes del vencimiento) y una llave de idempotencia <code>polizaId + periodo</code> .
7	El Módulo 1 pide versionamiento de APIs, pero los endpoints del Módulo 2 no llevan versión (<code>/polizas</code>).	Se respetan las rutas del enunciado. La versión se publica en el API Gateway (<code>/api/v1/polizas</code>), que reescribe hacia el servicio.
8	"Todas las acciones que modifiquen estados... deben consumir un servicio de edición". Si la llamada al CORE ocurre dentro de la transacción, un rollback posterior deja al CORE con un cambio que no existe.	El CORE se invoca después del commit (<code>@TransactionalEventListener(AFTER_COMMIT)</code>). Una falla del CORE no revierte la operación: se registra para reproceso. En la arquitectura objetivo esto es un outbox transaccional con reintentos y DLQ.
9	El Módulo 1 pide notificaciones para creación y renovación, pero el Módulo 2 no incluye el endpoint de creación.	La creación está en el diseño (Módulo 1). En el código se implementa solo lo pedido; las fábricas de dominio <code>Poliza.individual(...)</code> y <code>Poliza.colectiva(...)</code> ya validan sus invariantes.
10	Cancelar una póliza normalmente implica devolver la prima no devengada.	Fuera de alcance. Canon y prima se conservan como valores históricos y queda registrada la fecha de cancelación.
11	Módulo 3: el filtro <code>country = 'México'</code> depende de que el dato esté normalizado (tildes, mayúsculas). Si se corrige con <code>UPPER()</code> o <code>TRANSLATE()</code> en la consulta, el optimizador deja de usar el índice.	El país se normaliza a un código ISO (<code>MX</code>) o se usa un índice funcional. Ver Módulo 3.

Módulo 1 – Diseño de sistema

1.1 Arquitectura de alto nivel

La plataforma se organiza como un conjunto pequeño de microservicios alineados a capacidades de negocio, desplegados en Kubernetes (OpenShift, EKS o AKS según el estándar corporativo) en al menos dos zonas de disponibilidad. Cada servicio es dueño de sus datos y se comunica de dos formas:

- **Síncrona (REST)** para consultas y comandos que el usuario espera ver resueltos en la misma interacción: crear, consultar, renovar o cancelar.
- **Asíncrona (eventos en Kafka)** para todo lo que no debe bloquear al usuario ni acoplar la disponibilidad de la plataforma a la de terceros: sincronización con el CORE, notificaciones por correo y SMS, auditoría y proyecciones de lectura.

Componente	Responsabilidad
API Gateway	Punto único de entrada. Terminación TLS, validación de JWT (OAuth2/OIDC) y api-keys de aliados, rate limiting por consumidor, enrutamiento por versión (/api/v1), propagación de X-Correlation-ID . Detrás de un WAF.
Servicio de Pólizas	Dueño del agregado póliza: creación, consulta, modificación, renovación y cancelación. Calcula canon y prima. Publica PolizaCreada , PolizaRenovada , PolizaCancelada .
Servicio de Riesgos	Alta y cancelación de riesgos de pólizas colectivas y consulta de riesgos. Valida tipo y estado de la póliza con una proyección local alimentada por eventos, sin llamada síncrona por cada operación. Publica RiesgoAgregado y RiesgoCancelado .
Motor de Renovación	Job programado (Spring Batch + ShedLock) que selecciona las pólizas próximas a vencer, aplica las reglas de elegibilidad y ejecuta el caso de uso de renovación del Servicio de Pólizas de forma idempotente.
Servicio de Notificaciones	Consume eventos de negocio y envía correo y SMS mediante plantillas. Reintentos propios y registro de entregas. Una falla del proveedor nunca afecta el flujo transaccional.
Adapter CORE (capa anticorrupción)	Único componente que conoce el contrato del servicio de edición en WebLogic. Traduce eventos de dominio al formato del CORE, aplica circuit breaker, reintentos, idempotencia y envía a DLQ lo que no pudo procesar.
Bases de datos	Una base por servicio (PostgreSQL u Oracle según estándar), con réplica de lectura para consultas masivas. Redis como caché de consultas frecuentes.
Bus de eventos	Kafka con replicación 3, tópicos particionados por polizaId para preservar el orden de los eventos de una misma póliza. Schema Registry para versionar contratos.

Sobre separar Pólizas y Riesgos. El enunciado los lista como servicios distintos, pero el canon y la prima de una colectiva dependen de sus riesgos, así que en términos de DDD forman un mismo agregado. Separarlos obliga a consistencia eventual en la prima (el Servicio de Pólizas la recalcula al consumir RiesgoAgregado o RiesgoCancelado) y a validar reglas cruzadas con una proyección local. Es viable, y es lo que propongo para respetar el enunciado. Si el volumen inicial no justifica dos despliegues, empezaría con un **monolito modular** con los mismos límites de módulo, que después se separa sin reescribir el dominio. La solución del Módulo 2 sigue ese enfoque.

1.2 Patrones de arquitectura seleccionados

Patrón 1 – Arquitectura hexagonal (puertos y adaptadores)

Problema: las reglas de negocio de pólizas (cálculo de prima, renovación por IPC, cancelación en cascada, restricciones por tipo) deben mantenerse estables aunque cambie la forma de exponerlas o de integrarse. El CORE en WebLogic es la dependencia más volátil: hoy es un servicio SOAP o REST en capa media, mañana puede ser un CORE en la nube.

Solución: el dominio (agregado `Poliza`, entidad `Riesgo`, excepciones de negocio) no depende de frameworks de transporte ni de integración. La capa de aplicación orquesta casos de uso y habla con el exterior solo mediante puertos (`CoreEdicionPort`, `PolizaRepository`). Los adaptadores (REST, JPA, cliente HTTP del CORE, productor Kafka) se conectan en los bordes.

Beneficio concreto: reemplazar el CORE o pasar de llamada síncrona a outbox implica escribir un adaptador nuevo sin tocar las reglas. Las reglas se prueban con pruebas unitarias puras en milisegundos, sin levantar contexto de Spring. En el Módulo 2 hay 14 pruebas de dominio con esa característica.

Patrón 2 – Arquitectura orientada a eventos con Transactional Outbox

Problema: la plataforma debe operar 24/7, pero el CORE legado tiene ventanas de mantenimiento, latencias variables y capacidad limitada. Si cada operación depende de una llamada síncrona al CORE y a los proveedores de correo y SMS, la disponibilidad de la plataforma queda limitada por la del eslabón más débil.

Solución: cada cambio de estado se confirma en la base del servicio y, **en la misma transacción**, se inserta el evento en una tabla `outbox`. Un relay (Debezium CDC o un publicador programado) lo lleva a Kafka. El Adapter CORE y el Servicio de Notificaciones consumen de forma independiente, con reintentos y DLQ.

Por qué outbox y no publicar directo a Kafka: publicar después del commit puede perder el evento si el proceso cae entre ambos pasos, y publicar antes puede emitir un evento de algo que luego se revierte. El outbox elimina esa doble escritura inconsistente y garantiza entrega al menos una vez. Por eso los consumidores son idempotentes y deduplican por `eventId`.

Beneficio concreto: si el CORE está caído, las pólizas se siguen renovando y cancelando. Los eventos se acumulan y se aplican en orden cuando el CORE vuelve, y el estado de sincronización queda visible por póliza.

Patrón 3 – API Gateway con CQRS en el lado de consulta

Problema: hay varios canales (front de asesores, portal de inmobiliarias, aliados) con necesidades distintas de seguridad y cuota. Además, el volumen de consultas (listados por inmobiliaria, estado de pólizas y riesgos) es varias veces mayor que el de escrituras y compite por los mismos recursos de la base transaccional.

Solución: el API Gateway centraliza autenticación, rate limiting, versionamiento, CORS y trazabilidad, y evita que cada microservicio reimplemente esas preocupaciones transversales. Detrás, las consultas se separan de los comandos: los comandos van al modelo transaccional normalizado y las consultas se sirven desde réplicas de lectura o vistas desnormalizadas (p. ej. "pólizas por tomador con conteo de riesgos activos") construidas a partir de los eventos y cacheadas en Redis.

Beneficio concreto: el tráfico de lectura escala de forma independiente y no degrada renovaciones ni cancelaciones. El gateway permite convivir `v1` y `v2` y aplicar límites por aliado sin despliegues de los servicios.

Patrones complementarios: **Anti-Corruption Layer** en el Adapter CORE, para que el modelo legado no contamine el dominio. **Saga coreografiada** para la creación de colectivas con muchos riesgos y el recálculo de prima entre servicios. **Circuit Breaker, Retry y Bulkhead** (Resilience4j) en toda integración saliente.

1.3 Modelo de datos principal

Modelo lógico del Servicio de Pólizas. Los terceros se normalizan porque una misma persona puede ser arrendatario en una póliza y arrendador en otra, y el CORE los identifica por tipo y número de documento.

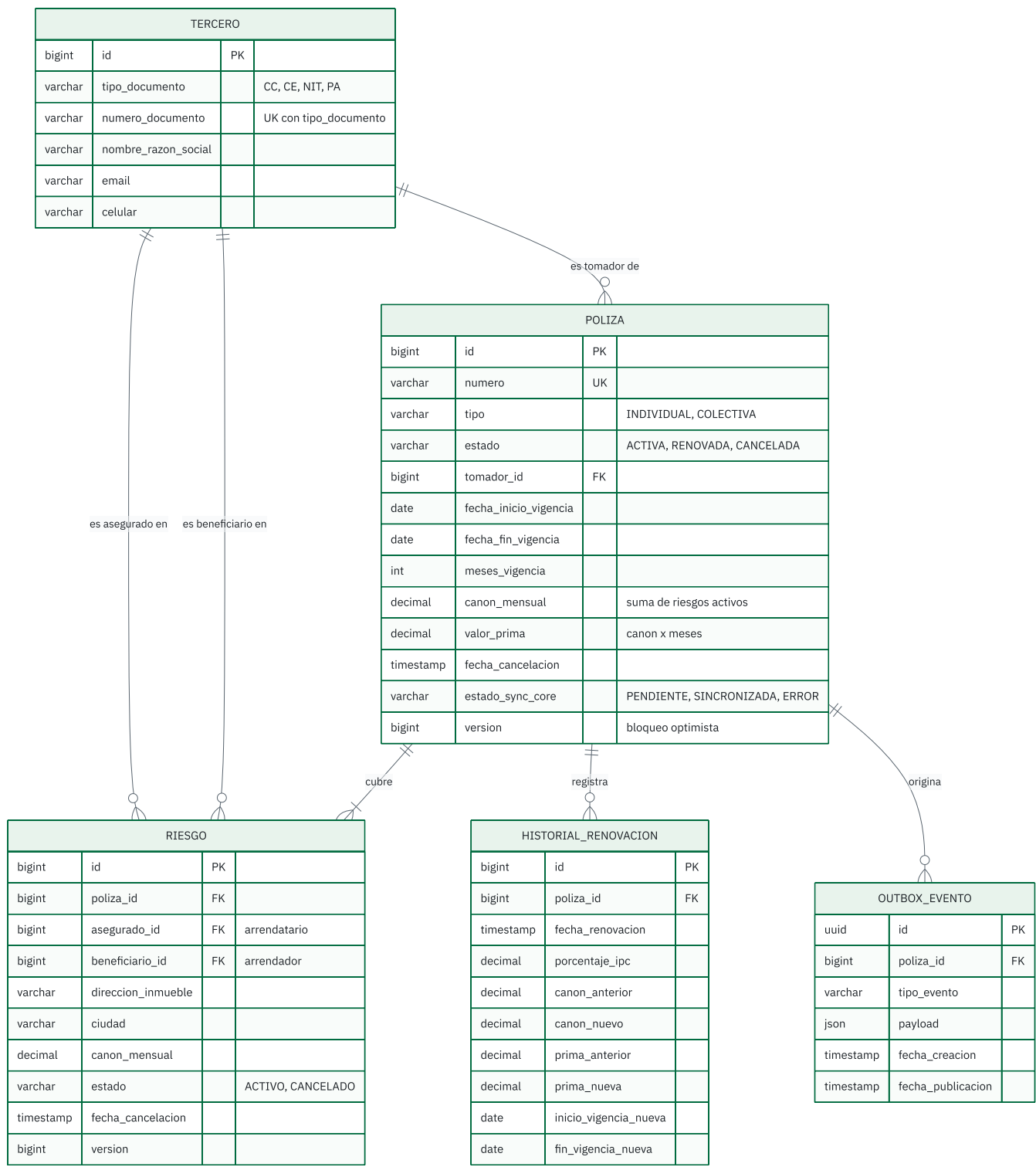


Figura 1. Modelo entidad-relación del Servicio de Pólizas.

Relación	Cardinalidad	Regla
Póliza – Riesgo	1 : N	Individual: exactamente 1 riesgo. Colectiva: 1 a N. Se garantiza en el agregado, porque en SQL estándar no se puede declarar un límite condicionado por el tipo.
Tercero – Póliza (tomador)	1 : N	Individual: el tomador es el arrendatario. Colectiva: la inmobiliaria o copropiedad.
Tercero – Riesgo (asegurado / beneficiario)	1 : N (x2)	Asegurado = arrendatario, beneficiario = arrendador. En la individual el asegurado coincide con el tomador.
Póliza – Historial de renovación	1 : N	Traza inmutable de canon, prima, IPC y vigencias de cada renovación.
Póliza – Outbox	1 : N	Eventos pendientes de publicar; se escriben en la misma transacción que el cambio de estado.

Índices principales: `poliza(tipo, estado)` para los listados, `poliza(fecha_fin_vigencia) WHERE estado <> 'CANCELADA'` para el motor de renovación, `riesgo(poliza_id)` y `tercero(tipo_documento, numero_documento)` único. Los montos se guardan como `DECIMAL(15,2)`, nunca como punto flotante.

1.4 Estrategias técnicas

Escalabilidad

- **Servicios sin estado** detrás del gateway, escalados con Horizontal Pod Autoscaler por CPU y por métricas propias (latencia p95, peticiones por segundo, lag del consumidor Kafka con KEDA).
- **Caché cache-aside en Redis** para consultas de póliza y listados por tomador, con TTL corto e invalidación por evento (`PolizaRenovada` y `PolizaCancelada` borran la llave).
- **Réplicas de lectura** y enrutamiento de transacciones `readOnly` hacia ellas. Pool de conexiones (HikariCP) dimensionado por pod para no saturar la base al escalar horizontalmente.
- **Kafka particionado por `polizaId`**: paralelismo entre pólizas y orden garantizado dentro de cada póliza.
- **Renovación masiva por lotes** con Spring Batch particionado (chunks de 500, particiones por rango de id), ejecutada fuera de horas pico, con ShedLock para que solo una réplica lance el job.
- **Paginación obligatoria** en listados, basada en keyset para colecciones grandes.

Logs y observabilidad

- **OpenTelemetry** como estándar único: agente Java para trazas automáticas de HTTP, JDBC y Kafka, exportadas a un collector y de ahí a Tempo o Jaeger (trazas), Prometheus (métricas) y Loki o ELK (logs).
- **Correlación de punta a punta**: el gateway genera X-Correlation-ID y `traceparent` (W3C). Viajan en headers HTTP y en headers de Kafka hasta el Adapter CORE, así una renovación se sigue desde el front hasta WebLogic. En el Módulo 2 esto está implementado con un filtro y MDC.
- **Logs estructurados en JSON** (formato ECS) con `correlationId`, `traceId`, `polizaId` y operación. Los datos personales (documentos, nombres) se enmascaran en cumplimiento de la Ley 1581 de 2012.
- **Métricas técnicas y de negocio**: RED por endpoint (tasa, errores, duración), estado de los circuit breakers, profundidad de la DLQ, lag de consumidores, pólizas renovadas por día, fallos de sincronización con el CORE.
- **SLO y alertas** sobre síntomas y no sobre causas: disponibilidad 99,9 % mensual, p95 < 300 ms en consultas, DLQ > 0 durante más de 15 minutos, eventos del outbox sin publicar con más de 5 minutos de antigüedad.

Tolerancia a fallos

- **Infraestructura:** mínimo 2 réplicas por servicio en zonas distintas, PodDisruptionBudget, probes de liveness y readiness, base de datos en alta disponibilidad con failover automático, Kafka con `replication.factor=3` y `min.insync.replicas=2`.
- **Integración con el CORE:** timeouts explícitos (conexión 2 s, lectura 5 s). Retry con backoff exponencial y jitter (3 intentos: 1 s, 2 s, 4 s) solo para errores transitorios (5xx, timeout). Circuit Breaker de Resilience4j (umbral de fallo 50 % en ventana de 20 llamadas, 30 s abierto). Bulkhead para que un CORE lento no agote los hilos.
- **Dead Letter Queue:** un mensaje que agota los reintentos pasa a `core.edicion.dlq` con la causa. La póliza queda con `estado_sync_core = ERROR` y un proceso de reproceso manual o programado lo reinyecta cuando se corrige la causa.
- **Idempotencia:** cada evento lleva `eventId`. El adapter lo envía como llave de idempotencia al CORE y los consumidores registran los ya procesados, así una reentrega no duplica ediciones ni notificaciones.
- **Concurrencia:** bloqueo optimista (`@Version`) sobre póliza y riesgo. Dos cancelaciones simultáneas no pueden dejar estados inconsistentes y la segunda recibe `409`.
- **Degradación controlada:** si Redis cae, se consulta la base. Si el proveedor de SMS falla, se reintenta por correo. Ninguna dependencia no crítica bloquea una operación de negocio.

Versionamiento de APIs

- **Versión mayor en la URI** (`/api/v1/polizas`): explícita, fácil de enrutar en el gateway y de leer en logs y métricas.
- **Solo los cambios incompatibles generan versión nueva** (eliminar o renombrar campos, cambiar semántica). Agregar campos opcionales o endpoints es compatible, y los clientes deben ignorar campos desconocidos.
- **Convivencia y retiro:** `v1` y `v2` conviven en el gateway. La versión que se retira responde con los headers `Deprecation` y `Sunset` y tiene un periodo de retiro acordado (p. ej. 6 meses), con métricas de uso por consumidor para saber quién falta por migrar.
- **Contratos:** OpenAPI versionado junto al código y pruebas de contrato dirigidas por el consumidor (Spring Cloud Contract o Pact) en el pipeline. Los eventos Kafka se versionan en Schema Registry con compatibilidad `BACKWARD`.

1.5 Diagrama de componentes

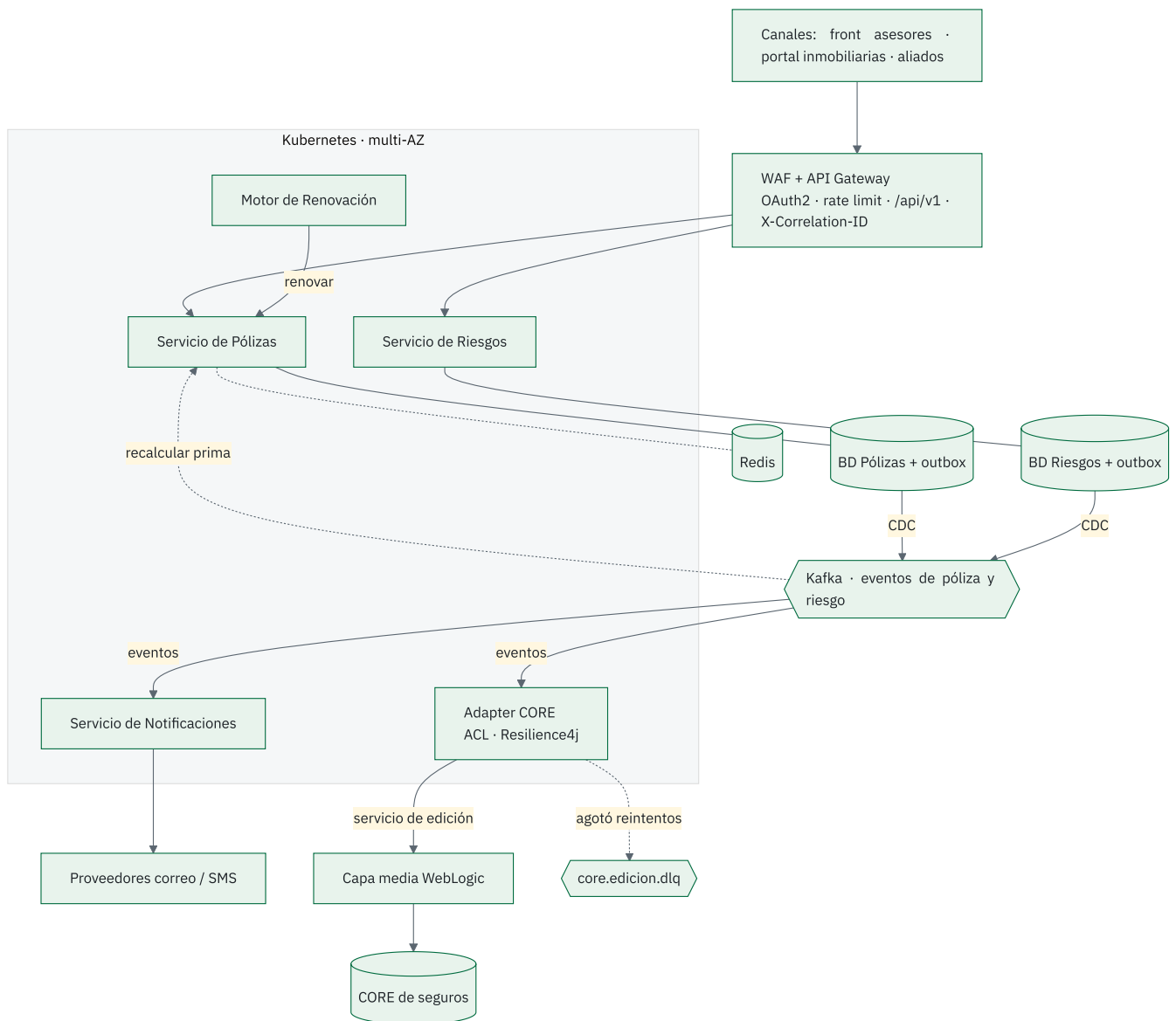


Figura 2. Componentes principales y sus integraciones.

Flujo de renovación automática

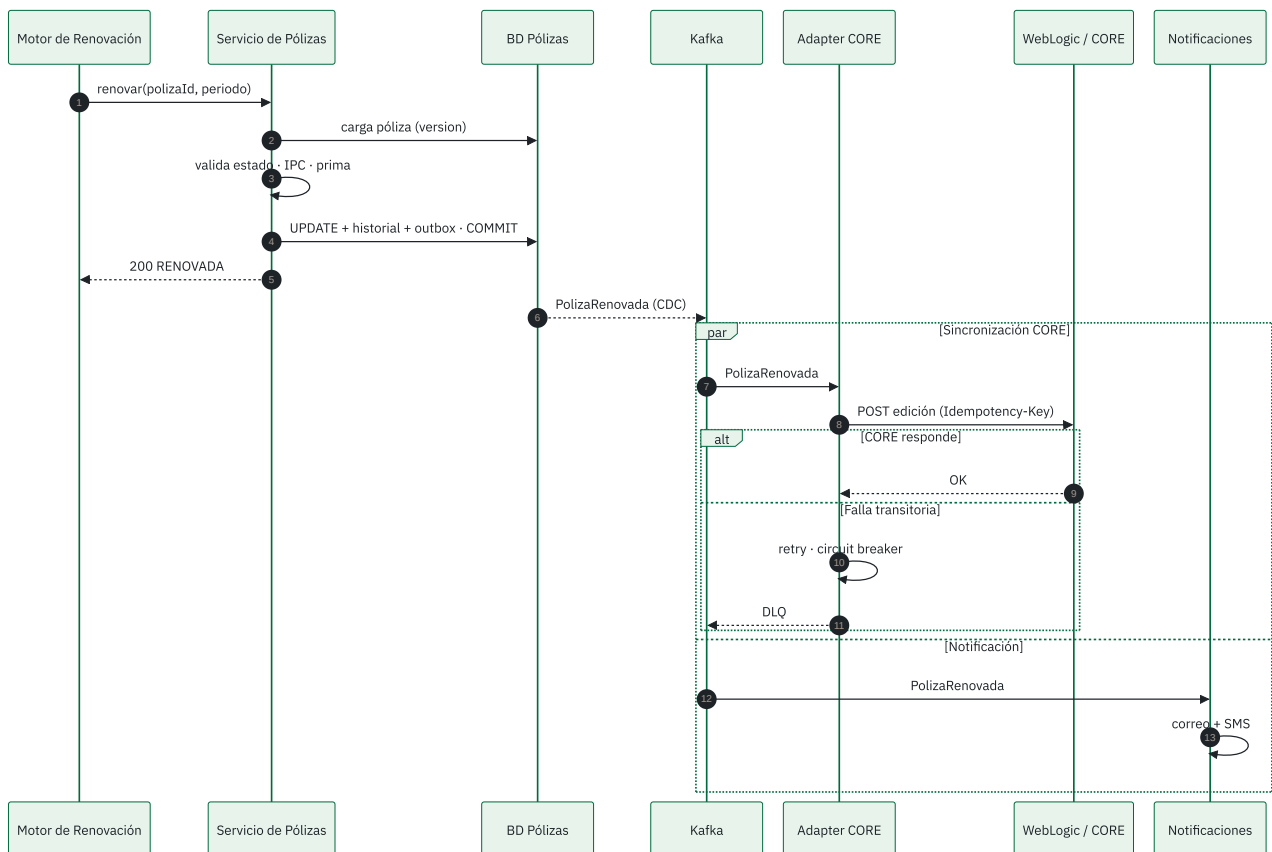


Figura 3. La operación se confirma una sola vez; CORE y notificaciones se resuelven de forma asíncrona e independiente.

Módulo 2 – Solución práctica: API de gestión de pólizas

El código completo, con instrucciones de ejecución y pruebas, está en el repositorio indicado en la portada. Esta sección resume las decisiones para facilitar la revisión.



Cada push a `main` ejecuta en GitHub Actions la compilación y las 34 pruebas (`./mvnw verify`). JaCoCo mide la cobertura y la publica en Codecov: **92,5 % de líneas** al momento de la entrega. En los pull requests se exige al menos 80 % sobre el código nuevo.

2.1 Stack

Java 17, Spring Boot 3.5, Spring Web, Spring Data JPA, Bean Validation, H2 en memoria con `schema.sql` y `data.sql`, Lombok, springdoc-openapi (Swagger UI), Actuator, JUnit 5, AssertJ, MockMvc, JaCoCo y GitHub Actions.

2.2 Estructura

```
src/main/java/com/segurosbolivar/polizas
├── api                      Adaptador de entrada REST
│   ├── controller          PolizaController, RiesgoController
│   ├── dto                 Requests/responses (records) con Bean Validation
│   ├── mapper              Traducción DTO ↔ dominio
│   └── error               GlobalExceptionHandler (RFC 7807)
├── application             Casos de uso
│   ├── port               CoreEdicionPort (puerto de salida)
│   └── service              PolizaService, RiesgoService, CoreSincronizacionListener
├── domain                  Núcleo, sin dependencias de infraestructura
│   ├── model              Poliza (raíz del agregado), Riesgo, Tercero, HistorialRenovacion
│   ├── exception          Excepciones de negocio con código estable
│   ├── event              PolizaModificadaEvent
│   └── repository          Puertos de persistencia
├── infrastructure          Adaptadores de salida y configuración
│   ├── persistence         Spring Data JPA
│   ├── core                Cliente HTTP del servicio de edición (WebLogic)
│   ├── web                 ApiKeyFilter, CorrelationIdFilter
│   └── config              Clock, OpenAPI
└── coremock                POST /core-mock/evento
```

Es hexagonal con una concesión pragmática: las entidades de dominio llevan anotaciones JPA. Separar entidad de persistencia y modelo de dominio duplica clases y mapeos sin beneficio real para este alcance. La frontera que sí se protege es la de comportamiento: **ninguna regla vive fuera de Poliza**, no hay setters públicos y los riesgos solo cambian de estado a través del agregado.

2.3 Endpoints

Endpoint	Éxito	Comportamiento y errores
GET /polizas?tipo=&estado=	200	Filtros opcionales y combinables. Valor inválido → 400 PARAMETRO_INVALIDO .
GET /polizas/{id}/riesgos	200	Riesgos activos y cancelados. Póliza inexistente → 404.
POST /polizas/{id}/renovar	200	IPC sobre el canon de cada riesgo activo, prima recalculada, nueva vigencia, estado RENOVADA , historial y notificación al CORE. Cancelada → 409 POLIZA_CANCELADA .
POST /polizas/{id}/cancelar	200	Póliza CANCELADA y todos sus riesgos CANCELADO , notificación al CORE. Ya cancelada → 409.
POST /polizas/{id}/riesgos	201	Solo colectivas; recalcula canon y prima. Individual → 422 TIPO_POLIZA_INVALIDO . Cancelada → 409. Cuerpo inválido → 400 con detalle por campo.
POST /riesgos/{id}/cancelar	200	Cancela solo ese riesgo y recalcula la póliza. Ya cancelado → 409 RIESGO_CANCELADO . Único activo → 409 ULTIMO_RIESGO_ACTIVO .
POST /core-mock/evento	202	Registra en el log el evento recibido.

Todas las rutas exigen `api-key: 123456`; sin ella o con un valor distinto responden `401` en formato `application/problem+json`. La comparación se hace en tiempo constante (`MessageDigest.isEqual`) y el valor se puede sobrescribir con la variable `API_KEY`. Swagger UI, la consola H2 y el health check quedan excluidos.

2.4 Decisiones relevantes

- **Integración con el CORE después del commit.** Los servicios publican `PolizaModificadaEvent`. Un `@TransactionalEventListener(AFTER_COMMIT)` invoca el puerto, cuyo adaptador hace un POST HTTP real a `/core-mock/evento` con `api-key` y `X-Correlation-ID`. Una operación que falla por regla de negocio nunca llega al CORE (está probado).
- **Errores RFC 7807** con `type`, `title`, `status`, `detail`, `instance` y las propiedades `codigo`, `timestamp` y `correlationId`. 404 para inexistentes, 409 para conflictos de estado, 422 para operaciones no permitidas por tipo y 400 para validación.
- **Dinero con BigDecimal**, escala 2 y `HALF_UP`. **Fechas con Clock** inyectado en zona `America/Bogota`.
- **Bloqueo optimista** con `@Version`; un conflicto concurrente responde 409 `MODIFICACION_CONCURRENTE`.
- **Pruebas:** unitarias del agregado (sin Spring), de integración con `MockMvc` por endpoint (reglas, códigos HTTP, notificación al CORE verificada con mock) y una de extremo a extremo sobre puerto real que confirma en el log que el adaptador llegó al mock.

2.5 Ejecución rápida

```
./mvnw spring-boot:run          # o: mvn spring-boot:run
./mvnw test

curl -s -H "api-key: 123456" "http://localhost:8080/polizas?tipo=COLECTIVA&estado=ACTIVA"
curl -s -X POST -H "api-key: 123456" http://localhost:8080/polizas/1/renovar
# Swagger UI: http://localhost:8080/swagger-ui.html
```

Módulo 3 – Optimización de la consulta orders / customers

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM   orders o
JOIN   customers c ON o.customer_id = c.customer_id
WHERE  c.country = 'México';
```

Uso sintaxis Oracle por la mención a PL/SQL. Al final de cada estrategia indico su equivalente en PostgreSQL.

3.1 Diagnóstico

Lo primero es medir y no suponer. Obtengo el plan real con estadísticas de ejecución:

```
SELECT /*+ GATHER_PLAN_STATISTICS */ o.order_id, o.order_date, c.customer_name, o.total_amount
FROM   orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE  c.country = 'México';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));
```

Con el esquema típico (solo llaves primarias), el plan esperado y sus causas son:

Síntoma en el plan	Causa
TABLE ACCESS FULL CUSTOMERS	No hay índice sobre <code>country</code> ; se leen los 500 mil clientes para quedarse con los de México.
TABLE ACCESS FULL ORDERS + HASH JOIN	<code>orders.customer_id</code> es llave foránea pero ni Oracle ni PostgreSQL indexan las FK automáticamente . Sin ese índice, la única forma de unir es recorrer los 10 millones de pedidos.
Hash join que se derrama a TEMP	La tabla hash no cabe en la PGA; aparecen lecturas y escrituras en temporal.
E-Rows muy distinto de A-Rows	Estadísticas desactualizadas o sin histograma en <code>country</code> , una columna muy sesgada, así que el optimizador estima mal la selectividad.

Dos preguntas antes de optimizar. (1) ¿Cuántas filas devuelve? Si México concentra el 40 % de los clientes, la consulta devuelve unos 4 millones de filas y ningún índice la hará "rápida": el problema pasa a ser de diseño funcional (paginar, agregar un filtro de fechas o precalcular). (2) ¿Cómo está guardado el país? Si coexisten `'México'`, `'Mexico'` y `'MEXICO'`, la consulta además es incorrecta. Si se "corrige" con `UPPER(c.country)` en el WHERE, se anula cualquier índice normal.

3.2 Estrategia 1 – Indexación dirigida: índices compuestos de cobertura

Idea: que la consulta se resuelva leyendo solo índices, sin visitar las tablas. El orden de las columnas importa: primero la columna del filtro o del join y después las columnas que la consulta proyecta.

```

-- Clientes: filtra por país y entrega customer_id (join) y customer_name (proyección).
-- El índice (country, customer_id) solo no cubre customer_name y obligaría a visitar
-- la tabla CUSTOMERS por cada cliente.
CREATE INDEX ix_customers_country_cov
  ON customers (country, customer_id, customer_name)
  ONLINE;

-- Pedidos: índice sobre la FK con las columnas proyectadas, para no visitar la tabla ORDERS.
-- Construcción paralela y online para no bloquear DML en producción.
CREATE INDEX ix_orders_customer_cov
  ON orders (customer_id, order_date, order_id, total_amount)
  ONLINE PARALLEL 8;

ALTER INDEX ix_orders_customer_cov NOPARALLEL;

-- Estadísticas con histograma en country, que tiene distribución sesgada.
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(
    ownname => USER,
    tabname => 'CUSTOMERS',
    method_opt => 'FOR ALL COLUMNS SIZE AUTO FOR COLUMNS SIZE 254 COUNTRY',
    cascade => TRUE);
  DBMS_STATS.GATHER_TABLE_STATS(
    ownname => USER,
    tabname => 'ORDERS',
    cascade => TRUE,
    degree => 8);
END;
/

```

Plan resultante esperado: INDEX RANGE SCAN IX_CUSTOMERS_COUNTRY_COV → NESTED LOOPS → INDEX RANGE SCAN IX_ORDERS_CUSTOMER_COV, sin ningún TABLE ACCESS. Si la proporción de México es alta, el optimizador puede preferir HASH JOIN sobre INDEX FAST FULL SCAN de ambos índices, que igual es mucho más barato que recorrer las tablas porque los índices son más angostos. El histograma es lo que le permite elegir bien.

Si el país no está normalizado: limpiar el dato es lo correcto (código ISO 'MX' con restricción CHECK o FK a un catálogo de países). Como medida temporal sirve un índice funcional que coincida exactamente con la expresión de la consulta:

```

CREATE INDEX ix_customers_country_norm
  ON customers (UPPER(TRANSLATE(country, 'ÁÉÍÓÚáéíóú', 'AEIOUaeiou')), customer_id,
customer_name);
-- La consulta debe usar la misma expresión:
-- WHERE UPPER(TRANSLATE(c.country, 'ÁÉÍÓÚáéíóú', 'AEIOUaeiou')) = 'MEXICO'

```

Costo: cada índice agrega trabajo a los INSERT y UPDATE sobre orders. En una tabla transaccional de alta escritura hay que medir la latencia de inserción antes y después. Aun así, el índice sobre la FK hace falta de todos modos para evitar bloqueos de tabla al borrar clientes.

PostgreSQL: CREATE INDEX CONCURRENTLY ix_orders_customer_cov ON orders (customer_id) INCLUDE (order_date, order_id, total_amount); y ANALYZE. La cláusula INCLUDE permite index-only scans; conviene un VACUUM para que el visibility map esté al día.

3.3 Estrategia 2 – Particionamiento con poda de particiones

Idea: que el motor lea solo el segmento físico de los clientes y pedidos de México. Hay un matiz importante: **orders no tiene columna de país**, así que particionarla "por zona geográfica" no es directo. Hay dos caminos:

a) Oracle: particionamiento por lista en CUSTOMERS y por referencia en ORDERS

El particionamiento por referencia hace que cada pedido herede la partición de su cliente a través de la FK, sin duplicar la columna de país. Un join filtrado por país poda particiones en ambas tablas.

```
CREATE TABLE customers_part (
  customer_id    NUMBER(12)    NOT NULL,
  customer_name  VARCHAR2(200) NOT NULL,
  country        VARCHAR2(60)  NOT NULL,
  CONSTRAINT pk_customers_part PRIMARY KEY (customer_id)
)
PARTITION BY LIST (country) (
  PARTITION p_mexico    VALUES ('México'),
  PARTITION p_colombia  VALUES ('Colombia'),
  PARTITION p_otros     VALUES (DEFAULT)
)
ENABLE ROW MOVEMENT;

CREATE TABLE orders_part (
  order_id       NUMBER(14)    NOT NULL,
  customer_id    NUMBER(12)    NOT NULL,
  order_date     DATE          NOT NULL,
  total_amount   NUMBER(14,2)  NOT NULL,
  CONSTRAINT pk_orders_part PRIMARY KEY (order_id),
  CONSTRAINT fk_orders_part_customer FOREIGN KEY (customer_id)
    REFERENCES customers_part (customer_id)
)
PARTITION BY REFERENCE (fk_orders_part_customer)
ENABLE ROW MOVEMENT;

CREATE INDEX ix_orders_part_customer ON orders_part (customer_id, order_date, order_id,
total_amount) LOCAL;

-- Migración de datos: en producción se hace online con DBMS_REDEFINITION
-- para no detener la operación; aquí la versión simple para ventana de mantenimiento.
INSERT /*+ APPEND PARALLEL(8) */ INTO customers_part SELECT customer_id, customer_name, country
FROM customers;
INSERT /*+ APPEND PARALLEL(8) */ INTO orders_part SELECT order_id, customer_id, order_date,
total_amount FROM orders;
COMMIT;
```

El plan muestra `PARTITION LIST SINGLE` sobre ambas tablas: se lee solo la partición de México.

b) Si la consulta siempre acota por fecha: rango por order_date

La consulta del enunciado no filtra por fecha, pero en la práctica "pedidos de clientes de México" casi siempre significa "del último mes o trimestre". Si es así, lo más efectivo es particionar `orders` por intervalo mensual y agregar ese filtro a la consulta:

```
CREATE TABLE orders_hist (
  order_id       NUMBER(14)    NOT NULL,
  customer_id    NUMBER(12)    NOT NULL,
  order_date     DATE          NOT NULL,
  total_amount   NUMBER(14,2)  NOT NULL,
  CONSTRAINT pk_orders_hist PRIMARY KEY (order_id)
)
PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH')) (
  PARTITION p_inicial VALUES LESS THAN (DATE '2024-01-01')
);

-- WHERE c.country = 'México' AND o.order_date >= ADD_MONTHS(TRUNC(SYSDATE, 'MM'), -3)
```

Costo: requiere licencia de Partitioning en Oracle Enterprise. Cambiar el país de un cliente mueve físicamente la fila y, en el caso (a), todos sus pedidos. Solo compensa en tablas grandes, y a 10 millones de filas ya empieza a ser relevante sobre todo para mantenimiento y archivado.

PostgreSQL: no tiene particionamiento por referencia. La alternativa es desnormalizar `country` en `orders` (mantenida por trigger o por la aplicación) y usar `PARTITION BY LIST (country)`, o particionar por rango de fecha.

3.4 Estrategia 3 – Vista materializada con refresco incremental

Idea: si la consulta se ejecuta con frecuencia y tolera unos minutos de desfase (reportes, tableros, conciliación), no tiene sentido recalcular el join de 10 millones de filas en cada ejecución. Se precalcula una vez y se refresca solo con los cambios.

```

-- Logs de cambios para permitir refresco rápido (FAST) en lugar de reconstrucción completa.
CREATE MATERIALIZED VIEW LOG ON orders
  WITH ROWID, SEQUENCE (order_id, customer_id, order_date, total_amount)
  INCLUDING NEW VALUES;

CREATE MATERIALIZED VIEW LOG ON customers
  WITH ROWID, SEQUENCE (customer_id, customer_name, country)
  INCLUDING NEW VALUES;

-- Una MV de join con refresco FAST requiere los ROWID de todas las tablas
-- y sintaxis de join en el WHERE.
CREATE MATERIALIZED VIEW mv_orders_customer_country
  PARALLEL 4
  BUILD IMMEDIATE
  REFRESH FAST ON DEMAND
  ENABLE QUERY REWRITE
AS
SELECT o.ROWID AS order_rid,
       c.ROWID AS customer_rid,
       o.order_id,
       o.order_date,
       o.total_amount,
       c.customer_id,
       c.customer_name,
       c.country
FROM   orders o, customers c
WHERE  o.customer_id = c.customer_id;

CREATE INDEX ix_mv_occ_country ON mv_orders_customer_country (country, order_date);

-- Refresco incremental cada 10 minutos.
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
    job_name      => 'JOB_REFRESH_MV_ORDERS_COUNTRY',
    job_type      => 'PLSQL_BLOCK',
    job_action    => q'[BEGIN DBMS_MVIEW.REFRESH('MV_ORDERS_CUSTOMER_COUNTRY', method =>
'F', atomic_refresh => FALSE); END;]',
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQ=MINUTELY;INTERVAL=10',
    enabled       => TRUE,
    comments      => 'Refresco incremental de pedidos por país');
END;
/

-- Consulta final: lee la MV por índice. Con QUERY_REWRITE_ENABLED = TRUE la consulta
-- original también se reescribe sobre la MV sin cambiar la aplicación.
SELECT order_id, order_date, customer_name, total_amount
FROM   mv_orders_customer_country
WHERE  country = 'México';

```

Costo: datos con hasta 10 minutos de desfase. No sirve para consultas transaccionales que necesiten el pedido recién creado. Los logs de MV agregan escritura en cada DML de las tablas base.

PostgreSQL: no tiene refresco incremental nativo. Se usa `REFRESH MATERIALIZED VIEW CONCURRENTLY` (requiere índice único en la MV) programado con `pg_cron`, o una tabla resumen mantenida por eventos.

3.5 Medidas complementarias

- **Paginación keyset** desde la aplicación (`WHERE (order_date, order_id) < (:ultima_fecha, :ultimo_id)`
`ORDER BY order_date DESC, order_id DESC FETCH FIRST 100 ROWS ONLY`), apoyada en el índice de

cobertura. Evita devolver millones de filas a un front que muestra 100.

- **Caché de resultados** en la aplicación (Redis) o `/*+ RESULT_CACHE */` en Oracle para lecturas repetidas con datos poco cambiantes.
- **Réplica de lectura** (Active Data Guard o réplica de PostgreSQL) para aislar esta carga analítica del OLTP.
- **Validar con evidencia:** comparar `buffer gets` , A-Rows y tiempo real antes y después con `DBMS_XPLAN.DISPLAY_CURSOR` , y fijar el plan con SQL Plan Baselines si es crítico.

Estrategia	Mejora esperada	Frescura	Costo principal	Cuándo usarla
Índices de cobertura	Alta si México es selectivo	Tiempo real	Escritura más lenta, espacio	Primera acción, siempre
Particionamiento	Alta en tablas muy grandes	Tiempo real	Licencia, rediseño, migración	Crecimiento sostenido, archivado
Vista materializada	Muy alta	Minutos de desfase	Refresco, logs de MV	Reportes y tableros frecuentes

Módulo 4 – Incorporar un fix de seguridad de main en feature/new-login

4.1 Respuesta: `git cherry-pick -x`

`git cherry-pick` toma el cambio introducido por un commit específico (su diff respecto al padre) y lo aplica como un **commit nuevo** sobre la rama actual. Es exactamente lo que pide el caso: traer solo el fix de seguridad sin el resto del historial de `main`.

- **Aislamiento:** no se mueve la base de la rama ni entra ningún otro commit de `main`.
- **Trazabilidad:** con `-x` el mensaje del commit nuevo incluye `(cherry picked from commit <hash>)`. En una auditoría de seguridad eso permite demostrar de dónde viene el cambio.
- **Sin reescribir historia:** no requiere `push --force`, así que es seguro en una rama compartida.
- **Convergencia futura:** cuando `feature/new-login` se integre a `main`, Git ve el mismo cambio en ambos lados y lo resuelve sin conflicto en la mayoría de los casos. Un rebase posterior sobre `main` descarta el commit duplicado porque identifica el parche equivalente (`patch-id`).

4.2 Procedimiento

```
# 1. Traer el estado actual del remoto sin modificar ninguna rama local
git fetch origin

# 2. Identificar el commit del fix (por mensaje, por archivo o por el PR fusionado)
git log origin/main --oneline --since="3 days ago"
git log origin/main --oneline --grep="security" -i
git log origin/main --oneline -- src/main/java/.../auth/
git show --stat <hash>           # confirmar que contiene solo el fix

# 3. Ubicarse en la rama de trabajo con el árbol limpio
git switch feature/new-login
git status
git stash push -u -m "wip new-login"    # solo si hay cambios sin confirmar

# 4. Aplicar el commit dejando registro de su origen
git cherry-pick -x <hash>

# 5. Si hay conflictos
git status                           # archivos en conflicto
#   editar y resolver conservando la intención del fix de seguridad
git add <archivos-resueltos>
git cherry-pick --continue
#   o, para volver al estado anterior sin rastro:
git cherry-pick --abort

# 6. Verificar antes de publicar
git log --oneline -3
git show HEAD
./mvnw verify                        # compilar y ejecutar las pruebas

# 7. Publicar (sin --force: no se reescribió historia)
git push origin feature/new-login

# 8. Recuperar el trabajo en curso si se guardó
git stash pop
```

Variantes según cómo se fusionó el fix en main

Situación	Comando
El PR se fusionó con squash (un solo commit)	<code>git cherry-pick -x <hash></code>
El fix son varios commits consecutivos A..B	<code>git cherry-pick -x A^..B</code> (incluye A)
El PR entró con merge commit	<code>git cherry-pick -x -m 1 <hash-del-merge> .</code> -m 1 indica que el diff se calcula contra el primer padre (main), de modo que se aplica exactamente lo que trajo el PR.
Se quiere revisar antes de confirmar	<code>git cherry-pick -x --no-commit <hash></code> , revisar con <code>git diff --staged</code> y luego <code>git commit</code>

Para comprobar después qué commits de la rama ya tienen equivalente en `main`: `git cherry -v origin/main feature/new-login`. Los marcados con `-` ya existen en `main` con otro hash.

4.3 Alternativas y por qué no son la primera opción

Alternativa	Qué hace	Evaluación para este caso
<code>git merge origin/main</code>	Integra todo lo que tiene <code>main</code> y la rama no.	Descartada: trae exactamente lo que se pidió evitar.
<code>git rebase origin/main</code>	Reescribe los commits de la rama sobre la punta de <code>main</code> .	Descartada: también incorpora todo <code>main</code> como nueva base. Además reescribe hashes y obliga a <code>push --force-with-lease</code> , lo que afecta a quien comparta la rama. Es útil para mantener la rama al día cuando sí se quiere todo <code>main</code> , no para traer un solo cambio.
<code>git diff <hash>^ <hash> git apply</code>	Aplica el parche como cambios sin confirmar.	Funciona, pero se pierde autoría y trazabilidad, y Git no puede reconocer luego que es el mismo cambio. Solo tiene sentido si hay que adaptar el parche a mano.
<code>git checkout origin/main --archivo</code>	Reemplaza el archivo completo por la versión de <code>main</code> .	Riesgosa: trae todos los cambios de ese archivo en <code>main</code> , no solo el fix, y puede borrar trabajo de la rama.
Rama <code>hotfix/*</code> desde el ancestro común	El fix se desarrolla en una rama creada desde <code>git merge-base main feature/new-login</code> (o desde el tag de producción) y se fusiona tanto en <code>main</code> como en las ramas que lo necesiten.	Es el flujo preventivo correcto para fixes de seguridad en Git Flow: cada rama recibe el fix con un <code>merge</code> limpio y sin commits duplicados. En este caso el fix ya está en <code>main</code> , así que <code>cherry-pick</code> es la respuesta. La recomendación para el equipo es adoptar este flujo en los próximos fixes críticos.

En GitHub: si la rama tiene PR abierto, el commit nuevo aparece en él y vuelve a disparar el pipeline de CI (pruebas y análisis de seguridad). Conviene dejar en la descripción del PR la referencia al PR original del fix. Si el cambio de seguridad es sensible, el PR original debería haberse gestionado como Security Advisory privado antes de hacerse público.

